

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

Verilog Assignment 2

August 2026

Instructions: This is a group assignment intended for practice. Each group must implement the Verilog design, write a suitable testbench, and demonstrate the working design and its behavioral simulation to a Teaching Assistant for evaluation.

Question 1: Binary–Gray Code Converter

Design and simulate a combinational circuit that converts a 4-bit binary number into its equivalent Gray code and converts the generated Gray code back into binary.

The complete design must contain the following Verilog modules.

1. Binary-to-Gray Converter

Create a module named

`binary_to_gray`

having the following interface:

Signal	Width	Direction
B	4 bits	Input
G	4 bits	Output

The module must convert the input

$$B = B_3B_2B_1B_0$$

into the corresponding Gray-code output

$$G = G_3G_2G_1G_0.$$

The conversion is defined by

$$G_3 = B_3,$$

$$G_2 = B_3 \oplus B_2,$$

$$G_1 = B_2 \oplus B_1,$$

$$G_0 = B_1 \oplus B_0.$$

2. Gray-to-Binary Converter

Create a second module named

`gray_to_binary`

having the following interface:

Signal	Width	Direction
G	4 bits	Input
B	4 bits	Output

The module must convert a 4-bit Gray-code input back into its corresponding binary value. The conversion is defined by

$$\begin{aligned} B_3 &= G_3, \\ B_2 &= B_3 \oplus G_2, \\ B_1 &= B_2 \oplus G_1, \\ B_0 &= B_1 \oplus G_0. \end{aligned}$$

3. Top-Level Converter Module

Create a top-level module named

`code_converter_top`

with the following interface:

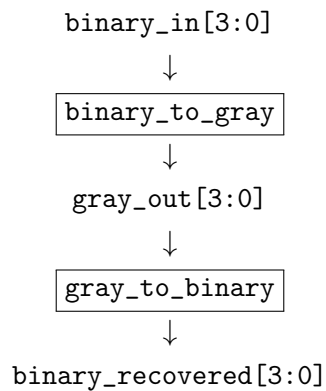
Signal	Width	Direction
binary_in	4 bits	Input
gray_out	4 bits	Output
binary_recovered	4 bits	Output

The top-level module must instantiate:

- one `binary_to_gray` module; and
- one `gray_to_binary` module.

The output of the Binary-to-Gray converter must be connected directly to the input of the Gray-to-Binary converter.

The required module connection is:



For every possible 4-bit input, the recovered binary output must satisfy

$$\boxed{\text{binary_recovered} = \text{binary_in}}$$

4. Example

For the binary input

`binary_in = 1011,`

the Binary-to-Gray converter should generate

`gray_out = 1110.`

Passing this Gray-code value through the Gray-to-Binary converter should produce

`binary_recovered = 1011.`

5. Verilog Design Requirement

The design must follow the modular hierarchy specified above.

Students may use suitable combinational Verilog constructs such as:

- gate primitives;
- continuous assignment using `assign`; or
- combinational `always @(*)` blocks.

The design must be purely combinational. No clock, flip-flop, latch, or other sequential logic is required.

6. Testbench and Simulation

Write a testbench for the top-level `code_converter_top` module.

The testbench must test **all 16 possible values** of the 4-bit binary input:

0000 through 1111.

For every test case, verify that

`binary_recovered = binary_in.`

The behavioral simulation waveform must show at least:

`binary_in,      gray_out,      binary_recovered.`

Demonstration is mandatory. Each group must demonstrate the working design and behavioral simulation to a Teaching Assistant. Students may be asked to explain the conversion logic and the connections between the three modules.

7. Submission

Submit the Verilog design source files and testbench in a single zipped folder named

`VerilogAssignment2_{Group_Number}.zip.`

The submission must contain:

- `binary_to_gray`;

- `gray_to_binary`;
- `code_converter_top`; and
- the testbench for `code_converter_top`.

Each source file must contain the group number, names and roll numbers of all group members, assignment number, and date of submission.